广电用户基本信息简单查询

- 查询广电用户的用户编号及开户时间
- 查询指定用户状态的用户基本信息
- 统计用户基本数据表中品牌名称的种类个数
- 统计不同用户等级名称的记录数
- 统计不同用户状态的记录数
- 统计指定用户数量范围的用户等级
- 统计用户数最多的3种用户状态
- 查询用户发生状态变更的时间及开户时间

一. 查询广电用户的用户编号及开户时间

- 掌握Hive的SELECT语句基本语法结构
- 使用SELECT语句从用户信息表中获取广电用户的用户编号及开户时间

入门SELECT语句

1. SELECT语句的语法结构

```
SELECT [ALL | DISTINCT] select_expr, select_expr, ...
FROM table_reference
[WHERE where_condition]
[GROUP BY col_list]
[HAVING having_condition]
[ORDER BY col_list]
[CLUSTER BY col_list
| [DISTRIBUTE BY col_list] [SORT BY col_list]
]
[LIMIT [offset,] rows];
```

上面是 SELECT 语句的完整语法模板，方括号 [ ] 内的子句均为可选，实际使用时按需组合即可。

子句说明

顺序	子句	作用
1	SELECT[ALL|DISTINCT]	选列；DISTINCT 去重
2	FROM	指定数据表
3	WHERE	行级过滤（**不能用聚合函数**）
4	GROUP BY	按字段分组
5	HAVING	对**分组后**的结果过滤（**可用聚合函数**，如 HAVING COUNT(*)>200）
6	ORDER BY	全局排序（所有数据进一个 Reducer）
7	CLUSTER BY / DISTRIBUTE BY+SORT BY	分布式排序/分发
8	LIMIT	限制返回行数（可带 offset 分页）

2. SELECT…FROM语句

```
SELECT sm_code, sm_name
FROM mediamatch_usermsg;
```

这条语句从 mediamatch_usermsg 表中查询并返回每一行记录的 sm_code（用户编号）和 sm_name（用户名称）两列，是最基础的 SELECT…FROM 查询。

3. 在SELECT…FROM语句中使用*通配符

```
SELECT *
FROM mediamatch_usermsg;
```

使用 * 通配符表示返回表中的所有列，无需逐一列出字段名，适合需要查看全部字段或尚不确定字段时使用。

4. 在SELECT…FROM语句中使用表别名

```
SELECT *
FROM mediamatch_usermsg AS `用户信息表`;
```

通过 AS 为表 mediamatch_usermsg 起了一个中文表别名 用户信息表（中文需用反引号包裹）。为表设置别名后，可在语句中用别名代替原表名，在多表查询时尤为方便。

任务描述

使用SELECT语句从用户数据表中获取广电用户的用户编号及开户时间

任务实现思路

- 广电用户基本数据表为mediamatch_usermsg，用户编号字段的名称为phone_no，开户时间字段的名称为open_time
- 根据SELECT…FROM语句的基本语法，将需要查询的字段的名称放在SELECT关键字之后，FROM关键字后面是查询的数据，即数据表mediamatch_usermsg

任务实现

查询广电用户的用户编号及开户时间

```
SELECT phone_no, open_time
FROM mediamatch_usermsg;
```

按任务要求，仅查询并返回 phone_no（用户编号）与 open_time（开户时间）两列。

二. 查询指定用户状态的用户基本信息

- 掌握Hive中的WHERE关键字的使用场景
- 使用WHERE关键字查询用户基本数据表中为“销户”状态的所有用户的用户编号、用户状态和开户时间

WHERE关键字的使用

1. 使用WHERE关键字添加查询条件

```
SELECT phone_no, open_time
FROM mediamatch_usermsg
WHERE phone_no = '2032485';
```

在 SELECT…FROM 基础上增加了 WHERE 条件，只返回 phone_no 等于 '2032485' 的记录，实现对行的过滤。

2. 使用关系运算符

语法结构：左操作数 关系运算符 右操作数

关系运算符	含义
=	判断两个操作数是否相等，相等则返回TRUE，否则返回FALSE
<>	判断两个操作数是否不相等，不相等则返回TRUE，否则返回FALSE
!=	同“<>”，判断两个操作数是否不相等，不相等则返回TRUE，否则返回FALSE
<	判断左操作数是否小于右操作数，小于则返回TRUE，否则返回FALSE
<=	判断左操作数是否小于等于右操作数，小于等于则返回TRUE，否则返回FALSE
>	判断左操作数是否大于右操作数，大于则返回TRUE，否则返回FALSE
>=	判断左操作数是否大于等于右操作数，大于等于则返回TRUE，否则返回FALSE

3. 使用IN关键字

```
SELECT phone_no, open_time
FROM mediamatch_usermsg
WHERE phone_no IN ('2116612','2017186','2017187','2082337','2212359','2212360');
```

使用 IN 关键字可一次性匹配多个取值：phone_no 只要等于括号中的任意一个值，该记录就会被返回，等价于多个 OR 条件。

4. 使用BETWEEN AND关键字

```
SELECT phone_no, open_time
FROM mediamatch_usermsg
WHERE phone_no BETWEEN '2032485' and '2032500';
```

使用 BETWEEN … AND … 筛选 phone_no 落在 '2032485' 到 '2032500' 闭区间内的记录，结果包含两端边界值。

5. 使用NULL关键字

```
SELECT phone_no
FROM mediamatch_usermsg
WHERE phone_no IS NOT NULL;
```

使用 IS NOT NULL 筛选出 phone_no 字段不为空的记录。注意 NULL 不能用 = 或 <> 比较，必须用 IS NULL / IS NOT NULL 来判断。

6. 匹配字符串有两种类别

```
SELECT phone_no, sm_name
FROM mediamatch_usermsg
WHERE sm_name LIKE '%宽频';
```

> *“%”通配符：%可匹配任意长度字符串，包括空字符串*

```
SELECT phone_no, sm_name
FROM mediamatch_usermsg
WHERE phone_no LIKE '229675_';
```

> *“_”通配符：匹配单个字符*

7. 使用RLIKE关键字·

```
SELECT phone_no, sm_name
FROM mediamatch_usermsg
WHERE sm_name RLIKE '.*(珠江|甜果).*';
```

RLIKE 是 Hive 中基于 Java 正则表达式的模糊匹配关键字，功能比 LIKE 更强大。其中 .* 表示匹配任意字符任意次，(珠江|甜果) 表示“珠江”或“甜果”二选一。整条语句的含义是：从用户表中筛选出 sm_name 字段里包含“珠江”或“甜果”的所有记录。

任务描述

用WHERE关键字查询用户基本数据表中为销户状态的所有用户的用户编号、用户状态和开户时间

任务实现思路

- 广电用户基本数据表为mediamatch_usermsg，
- 用户编号字段的名称为phone_no，用户状态字段为run_name，开户时间字段的名称为open_time。
- 根据SELECT…FROM语句的基本语法，SELECT关键字之后为要查询的字段名称，FROM关键字后面为数据表mediamatch_usermsg。
- 用户的销户状态分为主动销户及被动销户，需要对run_name字段进行模糊匹配，查询用户状态以“销户”结尾、起始字符内容未知的字符串的用户数据。

任务实现

查询指定用户状态的用户基本数据

```
SELECT phone_no, run_name, open_time
FROM mediamatch_usermsg
WHERE run_name LIKE '%销户';
```

按任务要求，用 `LIKE '%销户'` 模糊匹配 `run_name` 以“销户”结尾的用户状态，返回这些用户的编号、状态和开户时间。

三. 统计用户基本数据表中品牌名称的种类个数

- 掌握Hive中的`DISTINCT`关键字
- 掌握Hive中的聚合函数
- 使用聚合函数统计用户基本数据表中的品牌名称种类个数

使用`DISTINCT`关键字去重查询

1. `DISTINCT`关键字的语法结构

按品牌名称去重

```
SELECT DISTINCT sm_name
FROM mediamatch_usermsg;
```

该语句会对 `sm_name`（品牌名称）进行去重，即只保留品牌名称完全相同的记录中的一条，重复的记录不会重复出现。

简明对比表

名称	类型	核心特点	有无回看/点播
模拟有线电视	老式模拟信号	老闭路，雪花重影，已淘汰	❌ 无
数字电视	单向数字广电直播	数字直播，画质好，单向广播	❌ 无
互动电视	双向数字电视技术	数字直播+交互能力	✅ 有回看、点播
珠江宽频	广电宽带（上网）	同轴电缆上网业务，不是电视	—
甜果电视	广州广电互动电视品牌	4K机顶盒，广电双向网，含直播+点播	✅ 有回看、点播

简单理清关系

> *模拟有线电视 → 升级成单向数字电视；网络再升级双向 → 互动电视 广州广电把自家的互动电视叫甜果电视；同一条线的宽带叫珠江宽频，经常捆绑销售。*

按用户状态与用户等级名称去重

```
SELECT DISTINCT run_name, owner_name
FROM mediamatch_usermsg;
```

该语句会对 run_name 和 owner_name 两列的组合进行去重，即只保留用户状态与用户等级名称都完全相同的记录中的一条，重复的组合不会重复出现。

2. 使用聚合函数

```
SELECT MIN(phone_no)
FROM mediamatch_usermsg;
```

使用聚合函数 MIN() 返回 phone_no 列中的最小值。聚合函数会对整张表（或每个分组）进行计算，最终只输出一行汇总结果。

任务描述

使用聚合函数统计用户基本数据表中的品牌名称种类个数

任务实现思路

- 统计个数需要使用COUNT函数实现
- 由于品牌名称的种类有限，所以各条数据中的品牌名称可能存在重复情况
- 在聚合函数中，可使用DISTINCT关键字对数据执行去重操作

任务实现

统计用户基本数据表中品牌名称的种类个数

```
SELECT COUNT(DISTINCT sm_name)
FROM mediamatch_usermsg;
```

在 COUNT() 中使用 DISTINCT sm_name，会先对 sm_name（品牌名称）去重，再统计去重后的不同值个数，即品牌名称的种类数量。

四. 统计不同用户等级名称的记录数

- 了解Hive中的列别名使用场景
- 掌握Hive中的查询结果中如何显示列名
- 掌握Hive中列别名的语法
- 通过SELECT语句结合聚合函数COUNT统计不同用户等级名称的记录数

设置列别名

1. 列别名的使用场景

设置列别名能够优化查询语句，让查询语句及查询结果有更好的可读性。当原字段名为英文时，为方便查看查询结果，可以使用中文列别名代替英文字段名。

列别名常见的使用场景包括：

- 通过使用原数据列进行计算产生新的列，可以使用列别名为新产生的列命名；
- 调用了聚合函数等内置函数进行数据查询，可以使用列别名为函数的运行结果命名；
- 进行多表查询时，可能会出现不同表中存在相同列名的情况，为避免产生误解，应采用列别名进行区分。

列别名的使用规则：

- WHERE、GROUP BY、HAVING 等关键字中不能使用列别名；
- 进行排序操作时，则必须使用列别名，不能使用表达式。

2. 在Hive查询结果中显示列名

在Hive CLI中，使用SELECT语句得到的查询结果默认不进行列名的显示
开启列名显示的参数设置：

```
set hive.cli.print.header = true;
```

查询结果中的列名将以“表名.字段名”的格式显示
查询结果的列名中不显示表名的参数设置：

```
set hive.resultset.use.unique.column.names=false;
```

设置查询结果中列名的显示情况只在当前会话有效，再次登录后依然会使用默认设置显示查询结果

通过修改配置文件hive-site.xml中的内容可以使设置永久生效

```
<property>
<name>hive.cli.print.header</name>
<value>true</value>
</property>
<property>
<name>hive.resultset.use.unique.column.names</name>
<value>false</value>
</property>
```

> *Hive CLI 用 `set hive.cli.print.header=true`，Beeline 默认就有列名（或用 `!set showHeaders`）*

3. 列别名的语法结构

列别名的基本语法格式为 `[AS] alias_name`：

```
SELECT column_name
FROM table_name
```

这是列别名最简单的写法：`column_name` 为原字段名，`alias_name` 为其别名，二者之间可用 `AS` 连接，也可直接以空格隔开（`AS` 可省略）。
注意事项：

- 不能使用关键字作为列别名，否则会产生语法错误
- 可以使用中文设置列别名，但是中文需要使用反单引号（`）引起来

任务描述

通过`SELECT`语句结合聚合函数`COUNT`统计不同用户等级名称的记录数，为查询结果的`owner_name`字段设置别名“用户等级名称”，为聚合函数生成的设置列别名“记录数”。

任务实现思路

- 使用`GROUP BY`关键字根据`owner_name`字段进行分组
- 使用`COUNT`函数统计每个分组中的数据的记录数
- 在`owner_name`字段名及`COUNT(*)`表达式后需要添加列别名的定义

任务实现

统计不同用户等级名称的记录数

```
SELECT sm_name `品牌名称` FROM mediamatch_usermsg LIMIT 1;
```

为 sm_name 设置中文列别名 品牌名称，并加上 LIMIT 1 只返回一条结果，便于快速查看别名在查询结果中的展示效果。

```
SELECT owner_name `客户等级名称`, COUNT(*) AS `记录数`
FROM mediamatch_usermsg
GROUP BY owner_name;
```

按 owner_name 分组，用 COUNT(*) 统计每组记录数并起别名 记录数；同时把 owner_name 起别名 客户等级名称，使查询结果的中文列名更直观。

五. 统计不同用户状态的记录数

- 掌握Hive中的GROUP BY关键字
- 根据用户状态进行分组查询，统计不同用户状态的记录数

使用GROUP BY关键字分组查询

1. GROUP BY关键字语法结构

在Hive中使用GROUP BY关键字可以实现数据的分组查询。

不能在查询语句的SELECT关键字后使用GROUP BY引用字段以外的内容。

Hive要求SELECT关键字后的select_expr只能使用：

- GROUP BY使用的字段名组成的表达式
- 聚合函数

SELECT…FROM语句

```
SELECT select_expr, select_expr, ...
FROM table_reference
GROUP BY  col1, col2, …;
```

这是 GROUP BY 的语法骨架：GROUP BY 后跟一个或多个分组字段，查询结果会按这些字段的值进行分组。

分组查询1

```
SELECT phone_no, run_name, sm_name
FROM mediamatch_usermsg
GROUP BY run_name, sm_name;
```

按 run_name 和 sm_name 的组合进行分组。

注意：该例在 SELECT 中包含了一个既未聚合、也未出现在 GROUP BY 中的 phone_no，在 Hive 的 strict 模式或标准 SQL 下会报错；

分组查询2

规范写法应保证 SELECT 中的非聚合列都出现在 GROUP BY 中（见下方示例）。

```
SELECT run_name, sm_name
FROM mediamatch_usermsg
GROUP BY run_name, sm_name;
```

按 run_name 和 sm_name 组合分组，且 SELECT 中的列与 GROUP BY 完全一致，符合分组查询的规范写法。

分组聚合

```
SELECT COUNT(phone_no), run_name, sm_name
FROM mediamatch_usermsg
GROUP BY run_name, sm_name;
```

先按 run_name、sm_name 分组，再用 COUNT(phone_no) 统计每个分组内的记录数，即每种“用户状态 + 品牌”组合下的用户数量。

GROUP BY分组查询总结

- GROUP BY接续的部分是作为分组依据的字段名列表；
- GROUP BY与DISTINCT的作用机制类似；
- 若只有一个字段，则仅根据该字段的值进行分组；
- 若有多个字段，则所有字段值均相同的作为一组。

2. GROUP BY关键字与DISTINCT关键字

通过GROUP BY关键字查询到的结果与通过DISTINCT关键字查询到的结果是一样的，但两个关键字的作用不同：

- DISTINCT关键字仅有去除重复数据的作用；
- GROUP BY关键字的作用是对所有数据进行分组，再将每个分组的一条数据保留在最终的查询结果中。

使用GROUP BY关键字执行分组后还可以结合聚合函数进行统计分析，但是使用DISTINCT关键字后则无法执行进一步的统计分析。

任务描述

根据用户状态进行分组查询，统计不同用户状态的记录数

任务实现思路

使用run_name字段作为依据对mediamatch_usermsg表中的数据进行分组

使用COUNT函数统计每种用户状态的记录数

任务实现

统计不同用户状态的记录数

```
SELECT COUNT(*), run_name
FROM mediamatch_usermsg
GROUP BY run_name;
```

按 run_name 分组并用 COUNT(*) 统计每种用户状态的记录数，返回各状态及其对应的用户数。

六. 统计指定用户数量范围的用户等级

- 掌握Hive中的HAVING关键字
- 统计指定用户数量范围的用户等级，要求结果中只保留用户数大于200的用户等级

使用HAVING关键字对分组结果进行筛选

1. HAVING关键字的语法结构

WHERE关键字里不允许出现聚合函数。如果需要对聚合函数的执行结果进行筛选，那么需要引入一个新的关键字——HAVING。
语法结构

```
SELECT select_expr, select_expr, ...
FROM table_reference
GROUP BY  col1, col2, …
HAVING …;
```

这是 HAVING 的语法骨架：HAVING 必须跟在 GROUP BY 之后，用于对分组后的结果（可包含聚合函数）进行过滤。

Having示例

```
SELECT COUNT(*), run_name
FROM mediamatch_usermsg
GROUP BY run_name
HAVING COUNT(*) > 10000;
```

先按 run_name 分组并统计每组记录数，再用 HAVING 筛选出记录数大于 10000 的用户状态。HAVING 可以对聚合结果做条件过滤，这是 WHERE 做不到的。

任务描述

统计指定用户数量范围的用户等级，要求结果中只保留用户数大于200的用户等级

任务实现思路

- 将数据表mediamatch_usermsg中的数据按照用户等级名称owner_name字段进行分组
- 对分组后的数据使用COUNT函数进行计数统计
- 使用HAVING关键字对COUNT函数的执行结果进行筛选，只保留用户数大于200的用户等级

任务实现

统计不同用户等级名称的记录数

```
SELECT COUNT(*), owner_name
FROM mediamatch_usermsg
GROUP BY owner_name
HAVING COUNT(*) > 200;
```

按 owner_name 分组统计记录数，并用 HAVING COUNT(*) > 200 只保留用户数大于 200 的用户等级。

七. 统计用户数最多的3种用户状态

- 掌握Hive中的LIMIT关键字
- 掌握Hive中的ORDER BY、SORT BY、DISTRIBUTE BY、CLUSTER BY关键字
- 根据用户基本数据表统计出用户数最多的3种用户状态

使用LIMIT关键字设置查询结果展示

1. LIMIT关键字语法结构

在Hive中使用LIMIT关键字可以实现分页查询，主要用于：

- 设置查询结果从哪一条数据开始；
- 限制每次查询返回的数据条数。

SELECT...FROM...LIMIT语句

```
SELECT select_expr, select_expr, ...
FROM table_reference
LIMIT [OFFSET,] NUMBERS;
```

这是 LIMIT 的语法骨架：LIMIT 后可只写返回行数 NUMBERS，也可写成 OFFSET, NUMBERS 表示从第 OFFSET 条之后开始返回。

LIMIT示例1

查询mediamatch_usermsg表中的前两条记录。

```
SELECT *
FROM mediamatch_usermsg
LIMIT 4;
```

LIMIT 4 表示只返回查询结果的前 4 条记录，常用于快速预览数据。

LIMIT分页查询

- OFFSET表示偏移量，代表查询结果从哪条数据开始；
- 每次查询都是从第（OFFSET+1）条数据开始；
- OFFSET为可选参数，若不指定，则默认为0；
- NUMBERS表示执行查询语句后返回的数据条数。

LIMIT示例2

第一页

查询mediamatch_usermsg表中的两条记录，跳过前两条记录

```
SELECT *
FROM mediamatch_usermsg
LIMIT 0,2;
```

LIMIT 0,2 中第一个 0 是偏移量（跳过前 0 条），第二个 2 是返回行数，即返回第 1、2 条记录，实现分页效果。

第二页

查询mediamatch_usermsg表中的两条记录，跳过前两条记录

```
SELECT *
FROM mediamatch_usermsg
LIMIT 2,2;
```

LIMIT 2,2 中第一个 2 是偏移量（跳过前 2 条），第二个 2 是返回行数，即返回第 3、4 条记录，实现分页效果。

使用排序关键字对查询结果排序

1. Hive的排序关键字

Hive共有4个与排序相关的关键字

关键字	描述
ORDER BY	对**所有数据**根据指定排序字段进行排序
SORT BY	只针对分发到**一个Reducer中的数据**进行**局部排序**
DISTRIBUTE BY	控制Mapper的输出如何分发到Reducer中，不用于排序，一般与SORT BY结合使用
CLUSTER BY	将排序字段值相同的记录分发到一个Reducer中，然后针对一个Reducer数据进行局部排序，只能进行升序排列

2. ORDER BY关键字

Hive的ORDER BY关键字用于对查询到的所有数据根据某个排序字段进行排序

```
SELECT select_expr, select_expr, ...
FROM table_reference
ORDER BY col1[ASC|DESC], col2[ASC|DESC],…;
```

这是 ORDER BY 的语法骨架：可指定一个或多个排序列，每列后可加 ASC（升序，默认）或 DESC（降序）。

SELECT...FROM...ORDER BY排序语句

- 指定col1、col2等字段对查询结果进行排序；
- 若根据多个字段进行排序，则先根据第一个字段进行排序，若第一个字段值相同，则再根据第二个字段进行排序，以此类推；

- 每个排序字段名后都可以使用ASC或DESC参数，ASC代表升序排列，DESC代表降序排列；
- 排序参数若省略，则默认使用ASC进行升序排列。

分组与排序对比

	GROUP BY	ORDER BY
多列处理	逐级分组（列值全相等才归一组）	逐级排序（主列相同才看次列）
行是否被合并	✅ 每组只会输出 1 行（配合聚合）	❌ 每行都保留，只是排好序
结果性质	离散的"桶"	连续的"排列"

Hive中的ORDER BY

在Hive中使用ORDER BY关键字可以实现全局排序，将所有数据都放到一个Reducer里进行处理。对于大规模数据集，这个过程可能会导致执行时间过长

若将属性hive.mapred.mode的值设置为strict，则Hive要求ORDER BY关键字后必须使用LIMIT关键字限制查询数据的条数，默认情况下该属性的值为nonstrict

3. SORT BY关键字

在Hive中使用SORT BY关键字的语法与使用ORDER BY关键字的语法的区别仅在于关键字不同，其他内容完全一致，两者在执行结果上不同：

- ORDER BY关键字将所有数据进行全局排序；
- SORT BY只会针对分发到一个Reducer中的数据进行局部排序。
  当在排序过程中使用多个Reducer时，则最终输出的查询结果只是局部有序的，整体数据并不是有序的

4. DISTRIBUTE BY关键字

DISTRIBUTE BY关键字控制 Mapper 的输出如何分发到 Reducer 中：底层由分区器（Partitioner）计算 哈希值 % Reducer个数，把余数相同的记录分到同一个 Reducer，因此相同值的记录必然进入同一个 Reducer，但它本身不具备排序的作用。

- 在使用DISTRIBUTE BY关键字控制分发的过程中，可以继续使用SORT BY关键字对 Reducer 中的数据进行排序；
- DISTRIBUTE BY关键字与SORT BY关键字经常结合使用，使局部排序更有意义；

- 根据Hive的语法要求，`DISTRIBUTE BY`关键字需要写在`SORT BY`关键字之前。

示例

假设有 2 个 Reducer，示例数据为 1~6 这 6 个数字（为方便理解，这里直接用数值本身代替哈希值来演示取模过程）：

示例数据	哈希取模	分到的 Reducer
1	1 % 2 = 1	Reducer1
2	2 % 2 = 0	Reducer2
3	3 % 2 = 1	Reducer1
4	4 % 2 = 0	Reducer2
5	5 % 2 = 1	Reducer1
6	6 % 2 = 0	Reducer2

分发结果如下，奇数全部落到 Reducer1，偶数全部落到 Reducer2：

Reducer1得到的数据	Reducer2得到的数据
1 3 5	2 4 6

注意这种映射是多对一的：数据中再出现一个 `3`，它也一定进 Reducer1

在此基础上再使用 `SORT BY`，各 Reducer 内部各自升序，最终输出是 `1 3 5` 后接 `2 4 6`，只在 Reducer 内部有序，整体并不是 `1 2 3 4 5 6` 这样的全局有序。

确认 Reducer 个数

`DISTRIBUTE BY` 靠 `哈希值 % Reducer个数` 分发，所以只有 Reducer 个数 ≥ 2 时，分发才有意义。

Hive 默认按输入数据量估算 Reducer 个数（参数 `hive.exec.reducers.bytes.per.reducer`，默认约 256MB/个）。下面的示例经过 `WHERE` 过滤后数据量很小，Hive 往往只分配 1 个 Reducer；此时任何值取模都等于 0，所有数据必然进同一个 Reducer，`DISTRIBUTE BY` 完全失效 —— 输出和 `ORDER BY` 看起来一样，看不到任何分块。

因此跑示例前先手动指定 Reducer 个数：

```
set mapreduce.job.reduces = 3;    -- 旧版 Hive 写 set mapred.reduce.tasks = 3;
```

跑完后可从日志里的 number of reduce tasks 确认是否生效。

> *同理，单 Reducer 下 CLUSTER BY run_name, phone_no 与 DISTRIBUTE BY run_name SORT BY run_name, phone_no 的输出会一模一样 —— 这只是退化后的巧合，不代表两者语义相同：CLUSTER BY 会把分发键也变成 (run_name, phone_no)，一旦 Reducer 个数 ≥ 2，同一状态的记录就会被 phone_no 打散到不同 Reducer。*

DISTRIBUTE BY示例1

```
SELECT phone_no,run_name FROM  mediamatch_usermsg
WHERE sm_name='甜果电视'
DISTRIBUTE BY run_name
SORT BY run_name;
```

基础写法：先 WHERE 过滤出甜果电视的用户，再 DISTRIBUTE BY run_name 把相同状态分到同一个 Reducer，SORT BY run_name 在 Reducer 内做局部排序。

问题是 run_name 绝大多数都是“正常”，输出整屏几乎都是同一个状态，既看不到分块，也看不出排序。下面三个示例就是针对这一点逐步改造。

DISTRIBUTE BY示例2

```
SELECT phone_no, run_name
FROM mediamatch_usermsg
WHERE sm_name='甜果电视' AND run_name <> '正常'
DISTRIBUTE BY run_name
SORT BY run_name;
```

在 WHERE 里追加 AND run_name <> '正常'，先把占比绝大多数的“正常”筛掉。

WHERE 是 Map 端的行级过滤，先于分发执行，所以真正参与 DISTRIBUTE BY 分发的只剩剩下的几种状态，它们会各自聚成一段连续的块 —— “相同值进入同一个 Reducer”的分发效果这时就能看到了。

但 SORT BY 的排序列和分发列都是 run_name，块内每行的 run_name 完全一样，仍然看不出局部排序。

DISTRIBUTE BY示例3

```
SELECT phone_no, run_name
FROM mediamatch_usermsg
WHERE sm_name='甜果电视' AND run_name <> '正常'
DISTRIBUTE BY run_name
SORT BY run_name, phone_no;
```

在 SORT BY 中追加 phone_no 作为次级排序键，这样每个状态块内部的 phone_no 是升序的，局部排序终于可见。

注意它只在 Reducer 内部有序：不同状态块之间的先后顺序由各自的 Reducer 决定，整体并不是全局有序 —— 这正是 SORT BY 与 ORDER BY 的区别。

DISTRIBUTE BY示例4

```
SELECT phone_no, run_name
FROM mediamatch_usermsg
WHERE sm_name='甜果电视' AND run_name <> '正常'
DISTRIBUTE BY run_name
SORT BY run_name, phone_no
LIMIT 20;
```

最后补上 LIMIT 20。LIMIT 是整个查询的最后一步，只裁剪最终返回的行数，既不参与过滤、也不影响分桶分发（此时“正常”早在 WHERE 阶段就被去掉了），纯粹是让输出少一点、便于阅读和对比。

5. CLUSTER BY关键字

- CLUSTER BY关键字等价于DISTRIBUTE BY关键字和SORT BY关键字结合，相当于一种简写方法；
- DISTRIBUTE BY关键字和SORT BY关键字使用的列完全相同；
- 在排序过程中是升序排列。

CLUSTER BY示例1

```
SELECT phone_no, run_name
FROM mediamatch_usermsg
WHERE sm_name='甜果电视' AND run_name <> '正常'
CLUSTER BY run_name;
```

等价于 DISTRIBUTE BY run_name SORT BY run_name，也就是前面 DISTRIBUTE BY 示例2 的简写：相同状态的记录进入同一个 Reducer，Reducer 内按 run_name 升序排列，输出里每种状态聚成一段连续的块。

注意 CLUSTER BY 只能升序，不能写 DESC。

CLUSTER BY示例2

```
SELECT phone_no, run_name
FROM mediamatch_usermsg
WHERE sm_name='甜果电视' AND run_name <> '正常'
CLUSTER BY run_name, phone_no;
```

等价于 DISTRIBUTE BY run_name, phone_no SORT BY run_name, phone_no。

关键区别：多加的这一列会同时改掉分发键和排序键，因为 CLUSTER BY 强制两者完全相同。而 phone_no 基本每个用户都不一样，组合哈希后同一状态的记录会被打散到不同 Reducer ——“相同值进同一 Reducer”不再成立，每种状态会在输出中出现多次（每个 Reducer 一段），示例1 的分块效果反而没了。

所以想要“按状态分块 + 块内按号码有序”，只能用 DISTRIBUTE BY run_name SORT BY run_name, phone_no（DISTRIBUTE BY 示例3）—— CLUSTER BY 没法把分发列和排序列拆开，做不到这一点。

任务描述

根据用户基本数据表统计出用户数最多的3种用户状态

任务实现思路

使用run_name字段作为依据对mediamatch_usermsg表中的数据进行分组

使用COUNT函数统计每种用户状态的记录数

使用ORDER BY关键字对查询结果进行排序

使用LIMIT关键字对最终查询数据的条数进行限制，输出用户数最多的3种用户状态

任务实现

统计用户数最多的3种用户状态

```
SELECT COUNT(*) as nums, run_name
FROM mediamatch_usermsg
GROUP BY run_name
ORDER BY nums DESC
LIMIT 3;
```

先按 run_name 分组统计用户数（别名 nums），再按 nums 降序排列并取前 3 条，从而得到用户数最多的 3 种用户状态。

八. 查询用户发生状态变更的时间及开户时间

- 掌握Hive中的使用正则表达式模糊匹配查询字段名称的方式
- 掌握常见正则表达式的规则
- 在用户基本数据表中查询用户发生状态变更的时间以及开户时间

使用正则表达式查询数据

1. 正则表达式

正则表达式是一种文本模式，用于表达对字符串的一种过滤逻辑，描述在搜索文本过程中所匹配的一个或多个字符串。正则表达式是由大小写字母、数字和元字符组成的，其中元字符具有特殊的含义。

元字符	描述	元字符	描述
*	匹配前面的表达式任意次，包括0次	[abc]	字符集合，匹配a、b、c中的任意一个字符
+	匹配前面的表达式1次或多次	[^abc]	匹配除a、b、c之外的任意一个字符
?	匹配前面的表达式0次或1次	[a-z]	字符范围，匹配指定范围内的任意字符
.	匹配除“\n”和“\r”之外的任意字符	[^a-z]	匹配任何不在指定范围内的任意字符
a|b	匹配表达式 a 或者表达式 b		

2. 使用正则表达式选择要指定的列

在SELECT语句中使用单引号将正则表达式引起来，则会将正则表达式识别为普通字符串；

```
SELECT 'phone.*'
FROM mediamatch_usermsg;
```

这里的 phone.* 用单引号包裹，会被当作普通字符串而非正则表达式，因此这条语句只是把字面量 'phone.*' 作为一列常量返回，并不能按正则筛选列。

Hive 允许在反引号（`）中使用正则表达式来匹配列名（默认情况下反引号内的内容仅被当作普通标识符，正则不会生效），需要增加一个设置。

```
set hive.support.quoted.identifiers = none;
```

将该参数设为 none 后

```
SELECT `phone.*`
FROM mediamatch_usermsg;
```

反引号内的 phone.* 此时被当作正则表达式，表示返回所有以 phone 开头、后面跟任意字符的列（如 phone_no 等）。

任务描述

使用正则表达式在用户基本数据表中查询用户发生状态变更的时间以及开户时间

任务实现思路

通过观察字段名可以发现，这两个字段的字段名均以“_time”结尾，因此可以使用正则表达式进行字段名的模糊匹配。

任务实现

查询用户发生状态变更的时间及开户时间

```
SELECT `.*_time`
FROM mediamatch_userevent;
```

用正则 .*_time 匹配所有以 _time 结尾的列（如开户时间、状态变更时间等），从而一次性选出这些时间字段。

小结

本章介绍了SELECT语句的语法，对SELECT语句中的各个关键字的使用方法进行了详细介绍：

- 可以添加WHERE关键字对查询结果添加条件表达式，对查询数据进行过滤；
- 可以使用LIMIT关键字控制最终查询结果的数据条数；
- 可以使用GROUP BY关键字对数据进行分组，再对每组数据使用聚合函数进行统计分析；
- 可以使用ORDER BY、SORT BY、DISTRIBUTE BY、CLUSTER BY关键字实现对查询结果进行相应的排序。

要熟练掌握SELECT语句，需要在实际操作过程中多进行编码练习，培养数据思维能力。